```
"""Vantex paper bot: noise-area momentum + 5-min opening range breakout, shares,
Run `python3 bot.py loop` during market hours, or `python3 bot.py status`.
"""
import os, sys, json, math, time, requests, numpy as np, pandas as pd
from datetime import datetime, timedelta
from zoneinfo import ZoneInfo
ET = ZoneInfo('America/New_York')
_f = os.path.expanduser('~/.vantex/alpaca.env')
E = dict(l.strip().split('=', 1) for l in open(_f) if '=' in l) if os.path.exists
E = {k: os.environ.get(k) or E.get(k) for k in ('APCA_API_KEY_ID', 'APCA_API_SECR
if not (E['APCA_API_KEY_ID'] or '').startswith('PK'): sys.exit('Refusing to run:
H = {'APCA-API-KEY-ID': E['APCA_API_KEY_ID'], 'APCA-API-SECRET-KEY': E['APCA_API_
TRADE = 'https://paper-api.alpaca.markets'          # hard-wired to PAPER
DATA = 'https://data.alpaca.markets'
NA_SYMS = ['QQQ', 'AAPL', 'NVDA', 'AMZN', 'GOOGL', 'TSLA']
ORB_SYMS = ['QQQ', 'MSFT', 'NVDA', 'AMZN', 'TSLA']
SLEEVES = len(NA_SYMS) + len(ORB_SYMS)
LOG = os.environ.get('VANTEX_LOG', os.path.expanduser('~/vantex-bt/live_log.jsonl

def log(**k):
    k['ts'] = datetime.now(ET).isoformat(timespec='seconds'); print(json.dumps(k)
    open(LOG, 'a').write(json.dumps(k) + '\n')

def req(method, url, **kw):
    for i in range(6):
        r = requests.request(method, url, headers=H, timeout=30, **kw)
        if r.status_code == 429 or r.status_code >= 500: time.sleep(2 + 2 * i); c
        return r
    return r

def clock(): return req('GET', f'{TRADE}/v2/clock').json()
def equity(): return float(req('GET', f'{TRADE}/v2/account').json()['equity'])
def positions(): return {p['symbol']: float(p['qty']) for p in req('GET', f'{TRAD

def bars(sym, start, end, feed):
    out, tok = [], None
    while True:
        p = dict(timeframe='1Min', start=start.isoformat(), end=end.isoformat(),
        if tok: p['page_token'] = tok
        d = req('GET', f'{DATA}/v2/stocks/{sym}/bars', params=p).json()
        out += d.get('bars') or []; tok = d.get('next_page_token')
        if not tok: break
    df = pd.DataFrame(out, columns=['t', 'o', 'h', 'l', 'c', 'v', 'n', 'vw'])
    if len(df): df['t'] = pd.to_datetime(df.t).dt.tz_convert(ET); df = df.set_ind
```

```
    return df

def today_orders():
    start = datetime.now(ET).replace(hour=4, minute=0, second=0, microsecond=0)
    return req('GET', f'{TRADE}/v2/orders', params=dict(status='all', after=start

def sleeve_qty(orders, prefix):
    q = 0.0
    for o in orders:
        if (o.get('client_order_id') or '').startswith(prefix):
            q += float(o.get('filled_qty') or 0) * (1 if o['side'] == 'buy' else
    return q

def order(sym, qty, side, cid, **extra):
    if os.environ.get('DRY'):
        log(event='order_dry', sym=sym, side=side, qty=int(qty), cid=cid, **{k:st
    body = dict(symbol=sym, qty=str(int(qty)), side=side, type='market', time_in_
    r = req('POST', f'{TRADE}/v2/orders', json=body)
    log(event='order', sym=sym, side=side, qty=int(qty), cid=cid, status=r.status
    return r

def session_minutes(df):
    df = df.between_time('09:30', '15:59')
    df = df.assign(k=(df.index.hour * 60 + df.index.minute - 570))
    return df

# ---------------- noise-area momentum ----------------
def na_levels(sym, now):
    """sigma per minute from last 14 full sessions (SIP), today's open and prev c
    hist = session_minutes(bars(sym, now - timedelta(days=30), now - timedelta(mi
    days = sorted(set(hist.index.date))
    today = now.date()
    past = [d for d in days if d < today]
    full = [d for d in past if (hist.index.date == d).sum() >= 380][-14:]
    moves = []
    for d in full:
        g = hist[hist.index.date == d]; c = g.set_index('k').c.reindex(range(390)
        moves.append(np.abs(c.values / g.o.iloc[0] - 1))
    sig = np.mean(moves, axis=0)
    prev_close = hist[hist.index.date == past[-1]].c.iloc[-1]
    tdy = hist[hist.index.date == today]
    op = tdy.o.iloc[0] if len(tdy) else None
    return sig, prev_close, op

def na_step(now, slot_k, orders, pos, eq):
    for sym in NA_SYMS:
        try:
```

```
            sig, pc, op = na_levels(sym, now)
            live = session_minutes(bars(sym, now.replace(hour=9, minute=30, secon
            if op is None: op = live.o.iloc[0]
            live = live[live.k < slot_k]
            p = live.c.iloc[-1]
            tp = (live.h + live.l + live.c) / 3; vw = (tp * live.v).sum() / max(l
            s = sig[slot_k - 1]; ub = max(op, pc) * (1 + s); lb = min(op, pc) * (
            cur = sleeve_qty(orders, f'NA-{sym}-'); d = int(np.sign(cur))
            want = d
            if d == 0: want = 1 if p > ub else (-1 if p < lb else 0)
            elif d == 1: want = -1 if p < lb else (0 if p < max(ub, vw) else 1)
            else: want = 1 if p > ub else (0 if p > min(lb, vw) else -1)
            log(event='na_check', sym=sym, slot=slot_k, price=round(p, 2), upper=
            if want == d: continue
            tgt = want * math.floor(eq / SLEEVES / p)
            delta = tgt - cur
            if delta: order(sym, abs(delta), 'buy' if delta > 0 else 'sell', f'NA
        except Exception as e:
            log(event='error', where='na', sym=sym, err=str(e)[:300])

# ---------------- 5-min opening range breakout ----------------
def orb_step(now, orders, eq):
    for sym in ORB_SYMS:
        try:
            if any((o.get('client_order_id') or '').startswith(f'ORB-{sym}-{now:%
            b = session_minutes(bars(sym, now.replace(hour=9, minute=30, second=0
            f5 = b[b.k < 5]
            if len(f5) < 3: log(event='orb_skip', sym=sym, why='not enough bars')
            O, C, Hh, Ll = f5.o.iloc[0], f5.c.iloc[-1], f5.h.max(), f5.l.min()
            if C == O: log(event='orb_skip', sym=sym, why='doji'); continue
            d = 1 if C > O else -1; p = b.c.iloc[-1]
            stop = Ll if d == 1 else Hh; risk = abs(p - stop)
            if risk < 0.01 or (d == 1 and stop >= p) or (d == -1 and stop <= p):
            tgt = p + d * 10 * risk
            qty = math.floor(eq / SLEEVES / p)
            log(event='orb_signal', sym=sym, dir=d, price=round(p, 2), stop=round
            order(sym, qty, 'buy' if d == 1 else 'sell', f'ORB-{sym}-{now:%Y%m%d}
                  order_class='bracket', take_profit=dict(limit_price=str(round(t
        except Exception as e:
            log(event='error', where='orb', sym=sym, err=str(e)[:300])

def flatten():
    r = req('DELETE', f'{TRADE}/v2/positions', params=dict(cancel_orders='true'))
    log(event='flatten', status=r.status_code)

def status():
    a = req('GET', f'{TRADE}/v2/account').json()
```

```
    print('equity', a['equity'], 'last_equity', a['last_equity'], 'day P&L', roun
    for p in req('GET', f'{TRADE}/v2/positions').json():
        print(p['symbol'], p['qty'], 'avg', p['avg_entry_price'], 'now', p['curre

def due(now):
    m = now.hour * 60 + now.minute - 570
    return m

def loop(max_minutes=None):
    done = set(); t0 = time.time()
    while True:
        if max_minutes and time.time() - t0 > max_minutes * 60:
            log(event='handoff', why='runtime limit'); sys.exit(3)
        c = clock()
        if not c['is_open']:
            nxt = datetime.fromisoformat(c['next_open'])
            wait = (nxt - datetime.now(nxt.tzinfo)).total_seconds()
            if 'flat' in done or wait > 100 * 60:
                log(event='market_closed', next_open=c['next_open']); return
            time.sleep(min(max(wait, 5), 300)); continue
        now = datetime.now(ET); m = due(now)
        eq = equity()
        if 5 <= m < 15 and 'orb' not in done:
            orb_step(now, today_orders(), eq); done.add('orb')
        for k in range(30, 390, 30):
            if k <= m < k + 10 and f'na{k}' not in done:
                na_step(now, k, today_orders(), positions(), eq); done.add(f'na{k
        if m >= 385 and 'flat' not in done:
            flatten(); done.add('flat'); return
        time.sleep(20)

if __name__ == '__main__':
    cmd = sys.argv[1] if len(sys.argv) > 1 else 'status'
    if cmd == 'loop': loop(float(sys.argv[2]) if len(sys.argv) > 2 else None)
    elif cmd == 'status': status()
    elif cmd == 'na_now':   # one NA check at the most recent slot (catch-up)
        now = datetime.now(ET); m = due(now); k = (m // 30) * 30
        na_step(now, k, today_orders(), positions(), equity())
```